Security Relevant Changes in Reused Agent Skills

CSC 580 Advanced Software Engineering
University of Michigan-Flint, Fall 2026
Group 8 Final Project Report

Jim Prantzalos Kyle Cantrell Vaisnavii Mohanraj Christian Heiney

November 8, 2026

Abstract

AI coding agents use reusable packages of instructions and supporting files, called *agent skills*, to perform specialized tasks. Reuse saves development effort, but a modified skill may introduce commands, scripts, network operations, or external dependencies that an adopter did not expect. This raises a software supply-chain concern: security-sensitive instructions can enter a project through material developed elsewhere. GitSkills is a large collection of agent-skill files found in public GitHub repositories. Its repeated and related artifacts make it useful for studying how these instructions change.

We investigated whether related skill artifacts contain security-sensitive operations absent from a plausibly earlier observed form. Our static-analysis pipeline identifies related skills, infers a comparison direction from available evidence, compares their instructions and supporting files, and assesses changes against eleven defined risk indicators. Across three non-overlapping, stratified random trials, we analyzed **300 skill families**, 6,570 artifact occurrences, and 2,157 primary directed comparisons. The scanner identified supported indicators in **63 families (21.0%)**. Changes involving local script execution (39 families) and external code or dependency acquisition (26 families) were the most widespread. These automated findings identify changes worth reviewing, not confirmed vulnerabilities or verified copying histories. The sample rate does not estimate prevalence across all GitSkills, and independent human validation is in progress.

Introduction

AI coding agents can use reusable instructions, known as *agent skills*, to perform specialized work. Under the Agent Skills specification, a skill includes a `SKILL.md` instruction file and may also contain scripts, reference documents, and other supporting resources (Agent Skills, n.d.). Developers can adopt and adapt these packages instead of writing every workflow from scratch.

That convenience also creates a trust question. *Software supply-chain security* concerns risks introduced through software, code, or other resources obtained from outside a project. An agent skill may instruct an agent to execute a script, install software, contact an external service, or access local files. A copied or modified skill can retain its familiar name and purpose while changing what the agent is asked to do. The change may be legitimate, but it can also introduce an operation the person adopting the skill has not reviewed. As a result, judging a skill only by its name or description can miss a meaningful security-sensitive change.

Studying these changes is not as simple as comparing numbered software releases. Skills are found in different repositories, and files with the same name are not necessarily related. GitSkills is a research dataset of agent-skill files collected from public GitHub repositories (Destefanis et al. 2026). Its many recorded occurrences make it possible to search for related skills and examine how their instructions and supporting resources differ. However, the dataset does not provide a verified history showing which skill was copied from which, and some historical or supporting-file information is missing.

Our research asks whether modified or reused skills introduce command execution, network access, file-system access, or other security-sensitive behavior absent from a plausibly earlier observed artifact. We investigate this by comparing stored skill content, establishing relationships and direction only where the available evidence supports them, and identifying operational changes that warrant review. We do not execute the skills or attempt to determine malicious intent. The goal is to give developers and security reviewers a reproducible way to focus on what changed and why it may matter.

Research question and study scope

> **Do modified or reused skills introduce command execution, file-system access, network access, or other risky behavior that was absent from an earlier version or source artifact?**

The study operationalizes this question as a comparison of stored instructions between related artifacts. An **artifact occurrence** is one skill at a repository and path in the dataset. A **skill variant** groups identical file content, while a **bundle variant** also accounts for the known state of sibling resources. A **candidate family** contains qualifying occurrences with the same exact name. Sharing a name defines a search space; it is not evidence of reuse by itself.

The unit of directional analysis is an ordered pair of related artifacts. Its endpoints are designated **base** and **derived** according to the available direction evidence. These terms identify the comparison orientation, not proven original authorship or direct copying. Equivalent and direction-unknown relationships are retained separately rather than treated as negative security findings.

The primary reported outcome is the **proportion of sampled families** with at least one directed comparison supporting a defined risk indicator. The pair-level outcome is secondary, because one family can contribute many related comparisons. The indicator assesses a particular introduced or changed operation, not merely a new keyword, a larger file, or a higher count of matches. For example, changing the source of an executed remote script can matter even if both artifacts contain exactly one download-and-execute command. Replacing a package manager for an otherwise equivalent installation need not introduce a new dependency-acquisition operation.

The group's expectation is that some related artifacts will contain observable security-sensitive changes, but that many differences will reflect routine adaptation, packaging, or legitimate functionality. The competing explanation is that a reported indicator identifies an intended feature rather than unwanted exposure. The study does not measure a causal effect of reuse, execute instructions, or attempt comprehensive security detection. The eleven indicators define the scope of the assessment; behavior outside that scope is not automatically a scanner error.

Background and related work

The Agent Skills specification describes a reusable instruction file (`SKILL.md`) with optional scripts, references, and other resources (Agent Skills, n.d.). This format helps explain what a skill may contain, but it does not establish that its instructions

are safe or have been executed. GitSkills provides the dataset for our study: it records agent-skill occurrences found in public GitHub repositories, including their locations and content hashes (Destefanis et al. 2026). Repeated occurrences make it possible to look for related skills, although the dataset does not provide a verified history of copying between them.

Recent security research has examined the risks present *within* agent skills. Liu and colleagues developed SkillScan to analyze vulnerable patterns, including data exposure, privilege escalation, and supply-chain concerns (Liu, Wang, et al. 2026). Another study combined static screening with behavioral verification to investigate malicious skills distributed through community registries (Liu, Chen, et al. 2026). These studies help establish why skill contents deserve security review. Our study asks a different question: **what security-sensitive operations appear or change between related skill artifacts?** Their detection rates cannot be compared directly with ours because the datasets, definitions, and validation methods differ.

Research has also begun examining how skills evolve. Huang and colleagues studied changes to skills using GitHub commit histories, including which components are modified and why (Huang et al. 2026). Our data does not contain a complete version history for each skill. We therefore use content similarity and an ordered set of direction clues to construct *plausible* comparisons, without claiming to recover the actual history of reuse.

Our contribution is a reproducible method that brings these concerns together: identify related artifacts, record what supports their ordering, compare their instructions and available supporting files, and assess specific changed operations against defined risk indicators. The goal is to identify changes worth reviewing, not to label entire skills as malicious or demonstrate that an instruction executed.

Dataset and cohort

Data source and unit of analysis

We use GitSkills version 1.0.0, released on Zenodo on August 10, 2026. GitSkills collected agent-skill files from public GitHub repositories in July 2026. The release contains 3,797,117 **file occurrences**, including repeated copies of the same content (Destefanis et al. 2026). A skill generally consists of a `SKILL.md` instruction file and may include supporting files—called **sibling files** in the dataset—such as scripts or reference material. We import the released SQLite data into a local DuckDB database for analysis.

Our starting unit is a **skill family**: qualifying artifacts with the same exact skill name, even when their descriptions differ. The shared name identifies candidates for comparison; it does not establish that they are copies of the same skill. Within a family, the similarity analyzer groups sufficiently related skill bodies into **clusters**. We then examine selected **directed comparisons** between related artifacts for which the available evidence supports a plausible earlier-to-later order. The relationship and direction methods are explained in the next section.

Preparing comparison candidates

The project builds an `artifact_groupings` table from the GitSkills records. An artifact qualifies when its **frontmatter** (the YAML metadata at the start of `SKILL.md`) parses as valid, its instruction body is longer than 50 characters, and its content, name, and description are present. A candidate family must have at least two qualifying occurrences with the same name. Descriptions are normalized for supporting subgroups, but differences in description do not split a family during similarity analysis.

Family size (artifact occurrences)	Per trial	Three trials
Low: 2	20	60
Medium: 3–15	55	165
Large: 16–100	20	60
X-Large: 101–500	4	12
Extreme: 501 or more	1	3
Total	**100**	**300**

Table 1: Family-size strata and allocation across the three trials

An earlier preparation audit retained 943,257 occurrences in 162,995 same-name families. These are **candidate-pool counts**, not the denominator for our study and not the number of families that passed the final sampling checks. Appendix A provides the preparation SQL and audit. The saved Sprint 3 sampling reports record the candidate pools used when each trial was drawn.

Selecting the three Sprint 3 trials

We analyzed **three non-overlapping, stratified random samples of 100 families each**. This is the final study population for the report; earlier Sprint 1 and Sprint 2 cohorts helped develop and test the method but do not contribute to the Sprint 3 results.

A family enters the random selection process only if it passes two checks (RDR-014):

1. **Distinct states:** At least two combinations of the main skill-file **hash** (an identifier derived from its exact content) and **sibling fingerprint** (a summary of known supporting-file contents) differ. Repeated copies of the same known state do not meet this requirement.
2. **Usable relationship:** The similarity analyzer can identify at least one primary directed comparison involving different main-file hashes or different *known* sibling hashes. A different hash alone does not establish relatedness or direction.

When a randomly drawn candidate fails the relationship check, the picker records the rejection and draws a replacement from the same size group. **Selection does not run the security scanner** and does not favor families with risk indicators. Families that fail eligibility are excluded from the study population, not counted as families with no risk.

We divided candidate families into five groups by their **total number of artifact occurrences**, including repeated copies. The allocation deliberately gives larger families more representation than they have in the candidate pool, so the study includes both ordinary and unusually large families without letting the largest ones consume the entire sample.

The trials use recorded random seeds **5801**, **5802**, and **5803**. Trial 2 excludes names selected in Trial 1; Trial 3 excludes names selected in both previous trials. Each trial saves its

`families.txt` and `sampling_report.json`, preserving the selection and rejection record. These are separate, non-overlapping samples, but the exclusions mean they are not independent draws from an unchanged pool.

The allocation is **not proportional** to the wider candidate population. Our unweighted percentages therefore describe these 300 selected, relationship-eligible families; they must not be presented as population-wide estimates for GitSkills. The number of eligible families in each stratum after relationship screening has not been established for population weighting.

What we analyzed

The three trials loaded **6,570 artifacts** with none skipped. Their similarity analysis selected **2,157 primary directed comparisons** in total. A *primary* comparison connects a derived candidate to at most one selected predecessor, using the graph method described in Section 6. We did not scan additional related directed edges in these trials, because doing so could count overlapping changes repeatedly within the same family (RDR-013). Equivalent or direction-unknown relationships are retained for relationship review but are not assigned introduced/removed security behavior.

The main outcome is **family-level frequency**: how many sampled families contain at least one comparison with a supported risk indicator. We also report pair-level frequency using all analyzed primary directed comparisons as its denominator. The two measures answer different questions: how widely a signal appears across families versus how frequently it appears among selected comparisons. Comparisons within a family may share artifacts and are not independent historical events.

Evidence availability and safe handling

The main skill bodies were available for all loaded artifacts, but commit dates and supporting-file content are not always available. A known empty sibling bundle is different from one whose contents could not be retrieved. We report unavailable files and partial evidence extraction rather than treating them as proof that no behavior exists.

All skills and bundled files are untrusted data. We do not execute their commands or scripts, install referenced dependencies, fetch URLs, or clone repositories to replace missing source content. The saved manifests identify the dataset, rules, code revision, and selected inputs; local numeric artifact IDs alone are not portable across database builds. The full data dictionary and reproduction paths are linked in Appendix E.

Method

Research approach and security scope

Our question asks whether related skills introduce security-sensitive behavior absent from an earlier observed form. GitSkills contains skill-file snapshots and some repository and commit metadata, but **not a complete version history or a verified record of which skill was copied from which**. Some commit dates are missing; a first commit at the current file path may also be later than the skill's true origin. We therefore infer plausible relationships and ordering before comparing behavior. We do not claim to reconstruct a verified family tree.

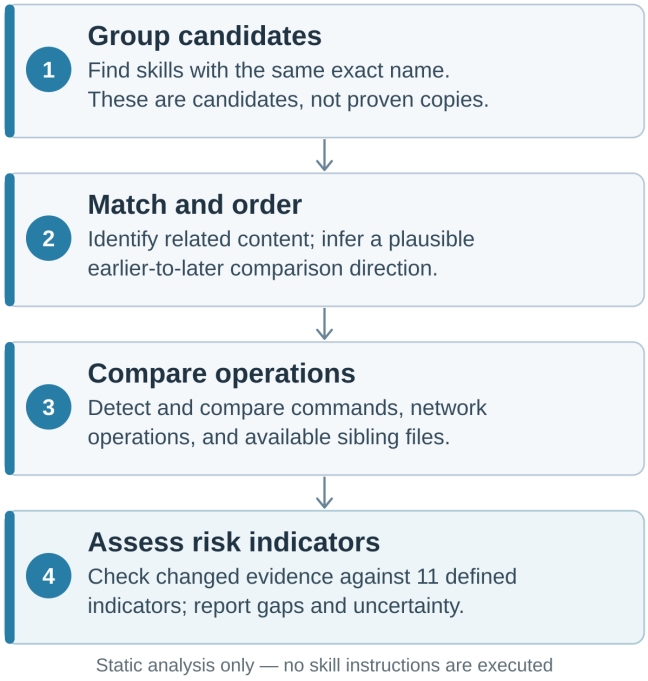


Figure 1: Four stages of the static comparison process: group candidates, match and order, compare operations, and assess risk indicators.

Our threat model focuses on observable instructions or code involving command execution, network access, file-system operations, credentials, external code or dependency acquisition, and system modification. These operations can be legitimate. They matter because an agent acting on untrusted instructions may have tools that affect a user's machine, data, or external services. A detected change is a **reason to review**, not evidence that an instruction ran, a vulnerability was exploited, or its author intended harm. The full threat model describes the trust boundary and coverage limits.

Analysis overview

The pipeline follows four stages. It first finds skills that *might* be related, then identifies sufficiently similar content and a defensible comparison order. Only after those decisions does it examine changed operations and assess them against our risk indicators. Separating these stages helps us avoid treating a shared name, a changed command, or a scanner match as proof of increased risk.

The next sections explain the evidence and limitations behind each stage.

Step 1: Identify related skills

We first compare artifacts within an exact-name candidate family (RDR-003). Identical `SKILL.md` content is recognized by its file hash; known bundled-file content is represented separately by a fingerprint of sibling file hashes. This matters because two copies of `SKILL.md` can contain the same instructions yet ship with different scripts. Unavailable sibling state is treated as unknown, not as an empty bundle.

For distinct main-file contents, we compare the body text after masking the opening YAML frontmatter. We normalize and tokenize the text, then form **five-token shingles**: sequences of five consecutive tokens. Comparing sets of these short sequences tolerates some formatting changes while retaining important command and identifier structure (RDR-004).

We require three signals of relatedness: at least **five shared shingles**, maximum **containment of 0.80**, and **Jaccard similarity of 0.30**. Containment asks how much of the smaller shingle set appears in the larger one; Jaccard measures the shared shingles relative to all distinct shingles across both texts. For example, if two files have 100 and 150 shingles, with 85 shared, containment is `85 / 100 = 0.85` and Jaccard is `85 / 165 = 0.515` (approximately 0.52). The first captures retained content in an expanded skill; the second guards against treating a small shared fragment as broadly similar. Using both is useful because the *added* material in a modified skill may contain the behavior we seek.

The analyzer groups related artifacts into **similarity clusters**. An exact shared name is therefore only a starting point: a large family can contain several clusters of differently written skills. The thresholds are documented operational choices, not probabilities of copying.

Step 2: Infer which artifact could come first

We assign an earlier (**base**) and later (**derived candidate**) only after establishing relatedness. The available evidence is tested in the following order (RDR-007):

For example, if a skill state has no recorded commit date but an equivalent copy was observed in January, that peer establishes that the state existed by January. If another related state is observed in March, January may support an earlier/later comparison. However, we cannot transfer that peer's date when its bundled-file state is different or unknown. Likewise, a repository created in March does **not** prove that its skill is newer than a skill in an older repository; a boundary is useful only when an independently dated state predates the repository.

When none of these methods supports direction, the relationship remains unresolved. Equivalent states remain peers rather than being forced into an earlier/later order. Each directed comparison retains its direction basis so that findings based on containment alone can be interpreted more cautiously than those supported by observed chronology.

The algorithm selects at most **one primary predecessor per derived candidate**, avoiding cycles. It can also identify other related, directed comparisons, called *additional edges*. RDR-013 selected **primary edges only** for the final three trials to study more families without repeatedly counting all related combinations in a few large families. In a controlled Sprint 2 comparison using the same scanner configuration, primary selection reduced 3,997 all-edge comparisons to 468 while retaining 43 of 44 observed family–indicator combinations. This supported the narrower design but did not establish that additional edges never contain unique findings. No separate Sprint 3 all-edge robustness result is claimed here.

Step 3: Detect and compare observable evidence

The static scanner applies **15 detection rules across six categories**: command execution, network access, file-system access, credentials, external code/dependency acquisition, and system modification. The rules identify recognizable constructs, such as a shell command, URL, file operation, or package-install instruction. They do not execute the content. The `SKILL.md` frontmatter is excluded from behavioral scanning because descriptive metadata and declared tools do not, by themselves, demonstrate an operation. Provenance metadata may still inform Step 2 (RDR-002).

We then **extract and compare evidence**, rather than interpreting a changed rule-match count as new risk. Evidence records retain the recognized operation, source location, context, and extraction completeness. The comparison distinguishes added, removed, and matching constructs. A different URL might matter even if the number of URL matches stays the same; repeated mentions can raise match counts without introducing an operation. Unclear associations are left uncertain (RDR-008 and RDR-012).

Bundled **sibling files** are assessed when added or changed content is available, regardless of file extension. For instance, a skill's instructions might remain the same while the bundled `setup.sh` changes. We examine the changed script's stored content, but do not infer its behavior from a filename when that content is missing. Known identical-content renames do not count as changed script behavior (RDR-009).

The comparator also distinguishes `skill-only`, `siblings-only`, `skill+siblings`, and `unknown` bundle changes. Here, **equivalent skill content** can mean equivalent under the token-shingle representation, not necessarily byte-for-byte identical; therefore, a `siblings-only` label does not by itself guarantee that no textual differences exist. Appendix B defines the classifications.

Step 4: Assess changes against risk indicators

The last stage tests the changed evidence against **11 risk indicators** (**RIs**) selected in RDR-010. Their purpose is to highlight observable ways in which an operation or source may change the agent's exposure to external code, networks, local files, credentials, or privileged actions. They do **not** assign a severity score. An indicator must be supported by the relevant *change* and its context; a rule match alone is insufficient.

We chose these indicators because they can be supported by **specific observable evidence** in the dataset rather than speculative runtime effects. They also reflect the plausible changes described in the threat model. An indicator may be supported, uncertain, or absent from the observed comparison. "Absent" means **not supported by available evidence**, not proven safe. One command may support several indicators without representing several separate incidents.

A useful negative example from scanner development is a change from `npm install` to `bun install` that preserves the same dependency-acquisition workflow. Both commands remain in the extracted evidence, but the substitution alone does not support **RI-005**. This is intentional: recording a changed command does not mean reporting a newly introduced risk (RDR-012).

Priority	Direction evidence	What it establishes—and what it does not
1	**Declared source**	Supported provenance metadata identifies a source; declarations can still be wrong or incomplete.
2	**Direct chronology**	`first_commit_at` records when a state was observed at its current file path, not its true original creation date.
3	**Equivalent-peer chronology**	A different occurrence of the *same known main-file and sibling state* supplies an earlier observed date when direct history is missing or later. The date belongs to the state, not automatically to the undated occurrence.
4	**Repository-creation boundary**	If one state was observed before another repository existed, that other repository occurrence could not have been earlier. Repository age alone never dates a skill.
5	**Directional containment**	When stronger evidence is missing, substantially greater containment in one direction (difference of at least **0.10**) can suggest a plausible base-to-expanded order. This is the weakest direction signal.

Table 2: Evidence used to infer earlier-to-later ordering, in priority order

ID	Security-sensitive change assessed
RI-001	Newly downloaded content is passed directly to an interpreter.
RI-002	The source URL of remotely executed code changes.
RI-003	A network operation uses a new hostname/domain or introduces explicit outbound transfer.
RI-004	HTTPS is downgraded to HTTP, or certificate verification is disabled.
RI-005	External code or dependency acquisition is introduced, or its source/registry changes. Package-name or version edits alone do not qualify.
RI-006	A destructive file operation is introduced or expands in scope or privilege.
RI-007	An operation newly accesses a recognized credential file or includes a credential reference directly in outbound transfer.
RI-008	An operation introduces or expands system, permission, service, or scheduled-task changes.
RI-009	A read/write operation newly targets a recognized sensitive file or directory.
RI-010	Local script execution is introduced or changes to a target with observably different content.
RI-011	A skill newly directs following external instructions, changes that source, or changes an ordinary reference into such a directive.

Table 3: Eleven evidence-based risk indicators

Worked example: A changed installation instruction

One comparison in Trial 3 illustrates the other side of that decision. In the `python-executor` family, the observed pair **1773238 → 1729578** changes the installation guidance:

Base artifact:

```
npx skills add inference-sh/skills@agent-tools
```

Derived candidate:

```
curl -fsSL https://cli.inference.sh | sh && infsh
↪  login
```

The base already describes software installation, so the finding is **not** that installation or network access became possible for the first time. The derived candidate introduces a *specific download-and-execute method*: it retrieves a remote shell script and passes it immediately to `sh`. It also introduces `cli.inference.sh` as a destination for an explicit network operation. The assessor consequently reported **RI-001** (direct download-and-execute) and **RI-003** (network expansion) from the same changed command—not two independent incidents.

This could be legitimate installation guidance, but executing remotely supplied code is relevant to supply-chain review because the contents of that code may change outside the skill. The stored instructions do not show whether an agent executed it, whether the agent's permissions would allow it, or whether the two artifacts share a verified copying history. The comparison has incomplete overall extraction coverage, although the installer evidence itself was extracted completely. **This case is an automated finding pending independent human validation.**

Outcomes and interpretation limits

Our **primary measure** is the number of sampled families with at least one supported RI divided by all sampled families. Secondary measures count primary directed comparisons containing at least one RI, the distinct families containing each indicator, and the distribution by family size, direction basis, and evidence coverage. Indicator frequencies can overlap, so their totals are not additive.

A **flagged comparison** may contain rule changes, extracted-evidence changes, or uncertain observations even if no RI is supported. A **supported finding** meets a defined indicator's automated criteria; it is not a validated security incident. We separately report partial extraction, missing sibling content, and uncertainty. Complete coverage means no known gap *within the implemented extraction scope*, not comprehensive detection of all risks. Appendix B contains the rule registry and analysis workflow; Appendix E identifies the source artifacts and saved outputs used for these results.

Results

Three-trial overview

Across three non-overlapping Sprint 3 trials, we analyzed **300 randomly selected, relationship-eligible skill families**, comprising **6,570 artifact occurrences** and **2,157 primary directed**

comparisons. The automated analysis found at least one supported **risk indicator** (**RI**) in **63 families** (**21.0%**). These are candidate security-sensitive changes identified under our rules, **not** confirmed vulnerabilities, malicious instructions, or observed executions. Independent manual validation is in progress.

The same code revision, rules, similarity policy, and primary-edge setting were used in all three trials. No selected artifacts were skipped. Because the study deliberately oversampled larger families and required usable directed relationships, these percentages describe **our selected families**, not all skills in GitSkills.

Finding 1: Security-sensitive changes appear across multiple families

The primary result is the proportion of sampled families with at least one supported indicator. That rate was **25% in Trial 1, 18% in Trial 2, and 20% in Trial 3**, or **21% overall**. The automated detections therefore appear across all three samples, rather than being confined to a single trial. Human review will determine how reliably those detections reflect the defined changes.

The family-level measure is intentional: a family contributes once whether it contains one indicator-bearing comparison or many. This keeps the main question focused on **how widely potentially important changes appear** rather than how often a highly reused family contributes another comparison. The effect of that choice is assessed below.

Finding 2: Script execution and external code acquisition are the most widespread changes

The risk indicators distinguish specific operational changes from ordinary differences in instructions. **RI-010** Local script execution was supported in **39 families** (**13.0% of all sampled families**), followed by **RI-005** External code or dependency acquisition in **26 families** (**8.7%**). Together, those two indicators appeared in **52 distinct families**: 39 had RI-010, 26 had RI-005, and 13 had both. Thus, **52 of the 63 indicator-bearing families** (**82.5%**) had at least one of these two indicators. The overlap matters; adding the individual counts would double-count families.

These categories **overlap**: one changed command or one family may support several indicators. For example, the `python-executor` installation change discussed in Section 6 supports both direct download-and-execute and a new network-operation destination. Neither observation establishes malicious intent or actual execution.

The distribution also varies by counting unit. **RI-006** Destructive or privileged file operations appeared in just **10 families**, but in **159 comparisons**; **134** of those comparisons came from `cso` and `mcp-builder`. That concentration is a reason to present family counts first. **RI-002**, **RI-004**, and **RI-009** had no supported detections in these trials. Zero detections do not establish that these behaviors are absent from other skills or undetectable by other methods.

Sensitivity check: Why families are the primary unit

The secondary comparison-level detection rate was **340 of 2,157 pairs** (**15.8%**), but it varied from **5.7% to 22.2%** across trials. A few families contributed many indicator-bearing comparisons: `cso` accounted for **55 of 113** in Trial 1, `junta-leiloeiros` for **8 of 36** in Trial 2, and `mcp-builder` for **133 of 191** in Trial 3. Excluding these three families *for a sensitivity calculation only* reduced the combined pair-level rate to **144 of 1,722** (**8.4%**), while the family-level rate changed only from **63 of 300** (**21.0%**) to **60 of 297** (**20.2%**). This supports using family-level frequency to describe the spread of detections in our sample; it does not remove sampling bias or prove population-wide risk prevalence.

Supporting observation: Family size and similarity clusters

Larger families were more likely to contain at least one supported indicator. They also offered more opportunities for directed comparisons and often contained more **similarity clusters**, which group artifacts with related instruction content under our thresholds.

For example, `implement` contained **856 artifacts in 35 clusters**, while `cso` had **150 artifacts in two clusters**. This reinforces why sharing a name is not enough to establish that skills are related. Multiple clusters do not prove different authors or independent origins, and a larger family provides more opportunities to detect changes. With only three Extreme families, their 100% detection rate should not be generalized.

Supporting observation: Not every detected difference supports a risk indicator

The scanner **flagged 1,119 of 2,157 comparisons** (**51.9%**), but only **340** (**15.8%**) met the narrower requirements for at least one supported indicator. The other **779 flagged comparisons** remain recorded as changes or uncertain evidence; they are **not** declared harmless. This separation is intentional. For instance, changing `npm install` to `bun install` may be a real command difference without introducing a new dependency-acquisition operation. The evidence comparison retains that difference, while the indicator assessment avoids inflating the security-sensitive count. Section 6 contrasts this with the `python-executor` download-and-execute example.

Evidence and ordering qualifications

Evidence coverage. Of the **2,157 comparisons**, **211** had incomplete evidence coverage, including **81 of the 340** with supported indicators. A particular detected operation can have complete extracted evidence even when another part of the comparison is incomplete. Among the **1,946 comparisons labelled complete**, **259** (**13.3%**) had supported indicators, compared with **340 of 2,157** (**15.8%**) overall. This is a coverage comparison, not a correction for detection error or selection bias.

Direction evidence. The 2,157 comparisons were oriented through declared source (10), direct chronology (399), equivalent-peer chronology (355), repository-creation boundaries (827), or directional containment (566). Of the 340 indicator-bearing comparisons, **131** relied on containment for

Measure	Trial 1	Trial 2	Trial 3	Combined
Families analyzed	100	100	100	**300**
Artifacts loaded	2,275	2,067	2,228	**6,570**
Primary directed comparisons	665	633	859	**2,157**
Families with a supported RI	25 (25.0%)	18 (18.0%)	20 (20.0%)	**63 (21.0%)**
Comparisons with a supported RI	113 (17.0%)	36 (5.7%)	191 (22.2%)	**340 (15.8%)**
Flagged comparisons (any reason)	346	271	502	**1,119**
Comparisons with incomplete evidence coverage	123	28	60	**211**

Table 4: Automated results from the three Sprint 3 trials

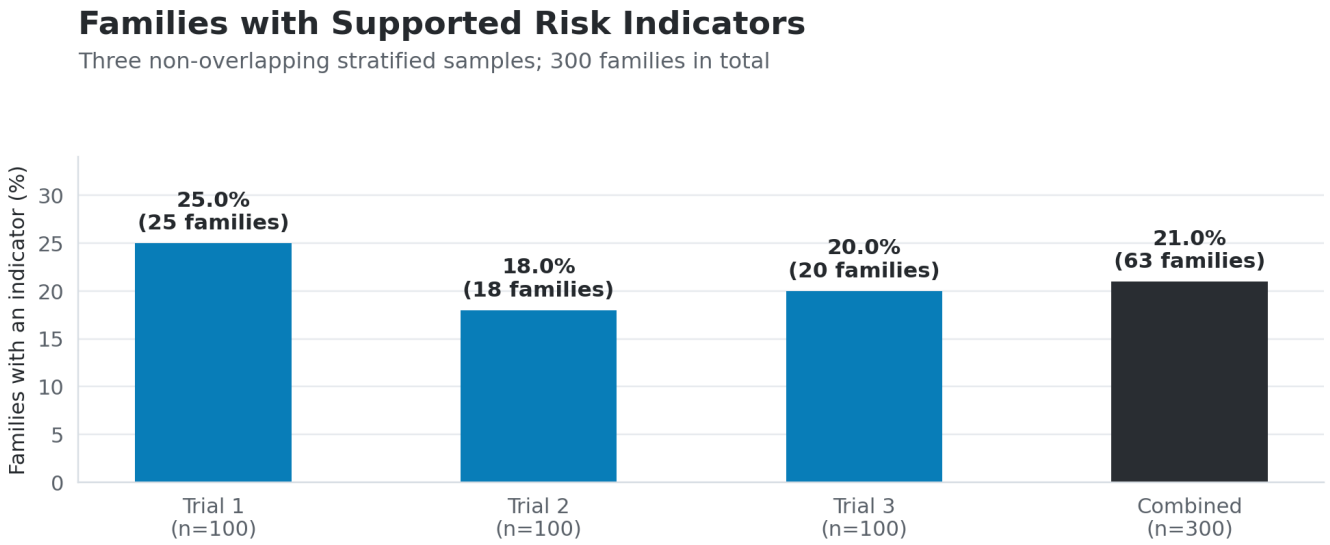


Figure 2: Share of sampled families with at least one supported risk indicator in each trial and overall.

Indicator	Families (of 300)	Comparisons (of 2,157)
RI-010 Local script execution	**39**	201
RI-005 External code/dependency acquisition	**26**	155
RI-003 Network expansion/redirection	**17**	52
RI-006 Destructive/privileged file operation	**10**	159
RI-008 System or permissions modification	8	18
RI-001 Direct download-and-execute	3	3
RI-007 Credential access/potential exposure	2	4
RI-011 External instruction dependency	1	1
RI-002, RI-004, RI-009	**0 each**	**0 each**

Table 5: Supported indicators by family and directed comparison

Trial	All comparisons	Without the listed dominant family
1 (`cso`)	113 / 665 = 17.0%	58 / 549 = **10.6%**
2 (`junta-leiloeiros`)	36 / 633 = 5.7%	28 / 613 = **4.6%**
3 (`mcp-builder`)	191 / 859 = 22.2%	58 / 560 = **10.4%**
Combined	**340 / 2,157 = 15.8%**	**144 / 1,722 = 8.4%**

Table 6: Sensitivity of comparison-level detection rates to three influential families

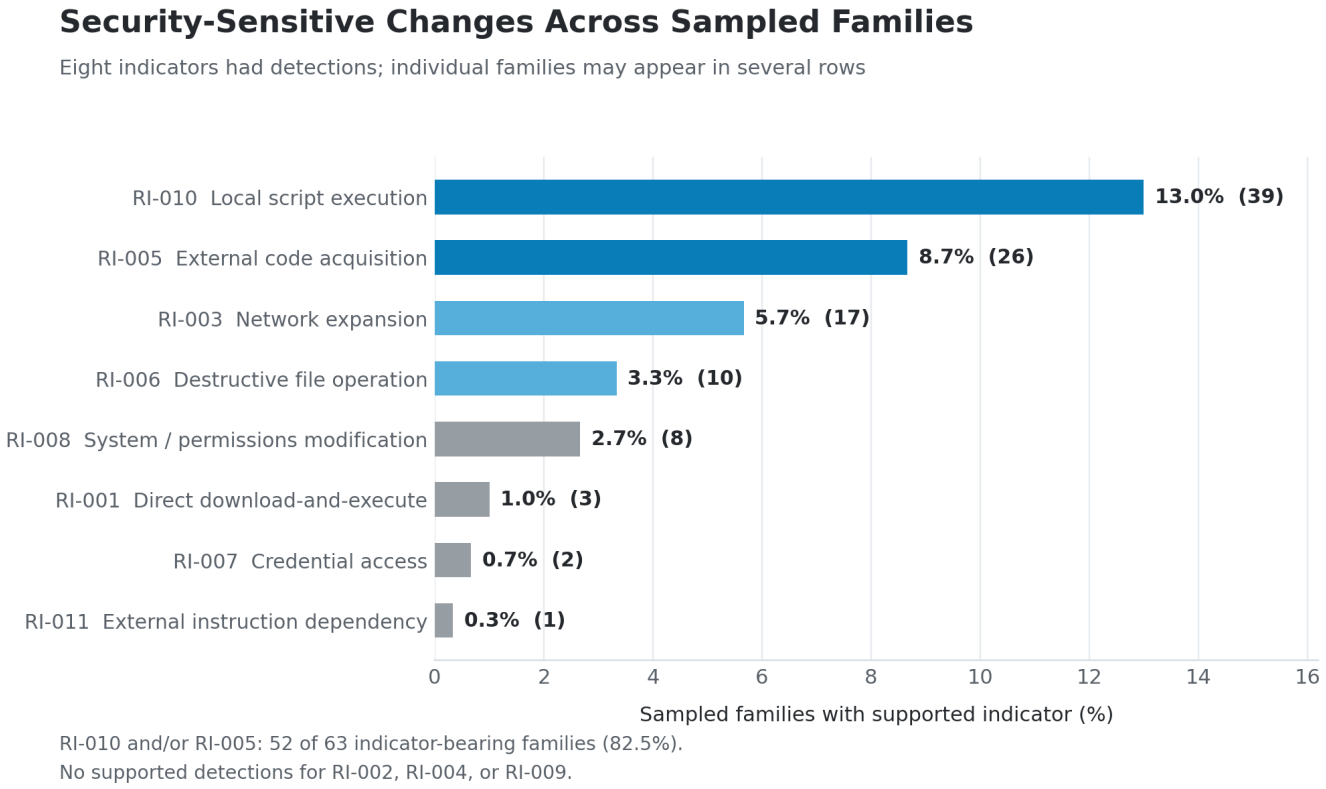


Figure 3: Number and percentage of sampled families with each of the eight detected risk indicators.

Family size	Families	Families with an RI	Rate	Mean similarity clusters per family
Low (2 artifacts)	60	4	6.7%	1.0
Medium (3–15)	165	24	14.5%	1.1
Large (16–100)	60	23	38.3%	1.5
X-Large (101–500)	12	9	75.0%	3.8
Extreme (501+)	3	3	100.0%	21.7

Table 7: Family size, indicator frequency, and similarity-cluster structure

direction. These methods support plausible earlier/later ordering, not verified histories of copying. Appendix D contains the full breakdown.

Supporting files. The analysis labelled **190 comparisons** `siblings-only` under its bundle-equivalence rules, with supported indicators in **41**. Most of those indicator-bearing comparisons are concentrated in `mcp-builder`, so manual review is needed before treating them as independent changes in executed-script behavior. Equivalence under the similarity representation does not guarantee byte-identical `SKILL.md` text.

The three trials' automated results are complete, but **independent human validation of relationship decisions and indicator judgments is still underway**. The next section describes how those findings will be evaluated.

Evaluation and validation

What the saved trials establish

The three Sprint 3 pipelines loaded all **6,570 selected artifacts**, produced auditable results for **2,157 primary directed comparisons**, and generated separate manual-review samples for each 100-family trial. Successful execution establishes that these outputs were produced under the recorded configuration; it does not establish that inferred relationships reflect historical copying, that every supported RI is correct, or that unflagged comparisons lack in-scope changes.

Automated tests and earlier constructed examples exercised the analysis logic, extraction, and output handling. Those checks are useful for regression testing, but they do not replace independent evaluation of the three final trials. The sample manifests, sampling reports, and validation inputs are retained separately to support reproducible review.

Manual-review design

Each trial has a **30-case directed sample**: five each from the supported-indicator, uncertain-indicator, coverage-limited, sibling-change, flagged-without-indicator, and unflagged-directed strata. Each also has a **15-case relationship sample**: five ambiguous, five equivalent, and five no-related-edge cases. All three use manual-review seed **580**, producing **90 directed cases and 45 relationship cases** overall. These are review subsets of the existing trial results, not additional families or new trial data.

Directed reviewers inspect the original base and derived bodies, relevant stored sibling files, evidence correspondence, and source support for each RI. Relationship reviewers separately assess whether the similarity method establishes substantive relatedness and whether the available evidence supports the suggested order. This distinction matters: a detector may correctly identify a changed command while an inferred earlier/later relationship remains contestable.

Independent review precedes team adjudication. AI-assisted case notes, where used, are not independent human judgments. Reviewed annotations must match the frozen identities and checksums in the saved sample; a changed family list or regenerated review sample cannot silently substitute for an earlier one.

Accuracy, uncertainty, and interpretation

For an RI, a **false positive** means the scanner reports a supported indicator that manual review finds unsupported under its defined conditions. A **false negative** means manual review identifies an in-scope supported indicator that the scanner did not report as supported. A missed additional command or rule match is not automatically a false-negative RI if the indicator category was already reported or the behavior falls outside its definition. Reviewers must also distinguish incomplete evidence from a confirmed absence of behavior.

The protocol limits indicator-accuracy calculations to comparisons assessable for the full scope: substantive relatedness, supported direction, adequately available evidence, and completed judgments on the eleven indicators. Other cases still inform error analysis and limitations. The sampled review strata intentionally include positive, negative, and difficult cases; their unweighted metrics cannot automatically be generalized to the entire GitSkills population.

Manual validation is underway. The saved three-trial summaries do not yet include completed independent and adjudicated accuracy or relationship-agreement results. We therefore do not report precision, recall, or agreement rates here. Once the team's adjudicated annotations are available, this section will be completed with those results, documented disagreements, and representative correctly and incorrectly classified cases. Appendix C describes the samples and Appendix E links their source artifacts.

Discussion

What the findings mean for skill reuse

A reused skill may look familiar while containing instructions or supporting files that have changed. That is the supply-chain concern behind our research question: **an adopter may encounter security-sensitive operations that were not present in a related earlier observed artifact**. Across the three trials, the scanner found candidate changes in **63 of 300 sampled families (21.0%)**. This shows that the defined indicators occur across the selected families, but it does not establish direct copying, malicious behavior, or actual execution. Independent manual validation is still underway.

The most widespread indicators involved **local script execution (39 families)** and **external code or dependency acquisition (26 families)**. At least one of these appeared in **52 of the 63 indicator-bearing families**. These changes matter because they can affect which scripts or outside code an agent is instructed to use. Their presence does not necessarily mean a skill is dangerous: the same instructions may be legitimate for its task, and an agent's permissions may limit what it can do. The relevant question for a reviewer is what operation changed, where it came from, and whether that change is appropriate in the intended environment.

Our evidence comparison helps narrow that review. The scanner flagged **1,119 comparisons**, but only **340** met the more specific criteria for a supported risk indicator. For example, switching package managers can change a command without introducing a new dependency-acquisition operation. The `python-executor` example in Section 6 shows the contrasting case: a later observed skill includes a direct remote-script execution command. Both examples illustrate why simply counting command or URL matches would overstate what the analysis can establish.

Why we report findings by family

We chose the family as our primary reporting unit because several comparisons can share artifacts or repeat the same evidence. The sensitivity check in Section 7 shows the effect: excluding one influential family from each trial changed the pair-level detection rate from **15.8% to 8.4%**, while the family-level rate changed only from **21.0% to 20.2%**. Family-level reporting better describes how widely these automated findings occur *within our sample*. It does not make the sample representative of all GitSkills or resolve uncertainty about relationships.

Practical value and remaining uncertainty

The pipeline is intended to support **review of changes**, not to assign a security score to a skill. It retains the detected operation, the supporting evidence, the basis for comparison direction, and any missing information so a reviewer can judge the finding in context. This could help developers and security reviewers prioritize changes to check when adopting or updating skills.

There are other reasonable explanations for many findings: ordinary feature additions, maintenance, different operating environments, or unrelated skills that share a name. In some comparisons, earlier/later ordering relies on weaker clues, and some supporting-file contents are unavailable. The manual reviews will test whether the selected relationships and indicator judgments hold up. Until then, the findings show **where the method identifies potential supply-chain concerns**, not how often those concerns become real security incidents.

Threats to validity

Construct validity. The eleven RIs define selected observable security-sensitive changes, not every security risk. Regular-expression rules and evidence extraction can mistake examples, warnings, or equivalent commands for behavior changes and may miss unfamiliar syntax, indirect operations, or instructions expressed only in prose. A larger number of matches is not automatically a higher level of risk. A complete evidence record means a recognized construct was recovered; it does not establish the behavior's runtime effect, safety, or intent.

Relatedness and historical order. Identical names can group unrelated skills; renamed relatives can be missed. Token-shingle containment and Jaccard support relatedness but do not prove copying, and similarity clusters do not identify verified original authors or sources. GitSkills does not supply complete historical contents. Commit dates refer to observations at current file paths; equivalent-peer chronology, repository-creation boundaries, and containment extend coverage but remain indirect. **566 of 2,157** directed comparisons use containment as their basis, including **131 of 340** comparisons with supported RIs. Their inferred order needs particular caution.

Missing evidence. Some sibling files have no stored text, and historical enrichment is uneven. The three trials report **211 comparisons with incomplete evidence coverage**, including **81** that also have supported indicators. A supported observation can still have complete local evidence when another part of its comparison is missing. Conversely, complete-labelled comparisons can contain behavior outside the rules. Unknown sibling state is not equivalent to an empty or unchanged bundle.

Sampling and generalizability. All three trial samples were stratified at 20/55/20/4/1 families by artifact-count group and required at least one usable primary directed relationship. Larger families were deliberately oversampled; candidate families rejected by the selection screen were not counted as having no findings. The 300 sampled families are non-overlapping but not independent draws from a constant pool because later trials exclude prior selections. Neither the **21.0% family-level rate** nor the **15.8% comparison-level rate** estimates prevalence among all GitSkills families. Family size measures artifact occurrences, not independent authors or historical modifications.

Dependence and conclusion validity. Multiple directed comparisons may share artifacts or repeat the same underlying evidence. For example, `mcp-builder` accounts for 133 of the 191 supported-indicator comparisons in Trial 3. Pair-level counts are therefore not independent incidents. The study analyzes primary predecessor edges only; additional directed comparisons could contain distinct observations that are not included. Trial-to-trial comparisons and dominant-family exclusions show sensitivity, but do not substitute for a population-weighted study or a matched all-edge robustness analysis.

Validation and reproducibility. The three trials generated 90 directed and 45 relationship-review cases, but their human adjudication is in progress. Scanner precision, recall, and relationship agreement are not established by the automated summaries or by AI-assisted draft notes. The pipeline preserves manifests, data identities, code revision, rules, seeds, and case checksums; numeric artifact IDs remain local to a particular DuckDB build. The saved manifests still mark approval as provisional. Reproducibility safeguards limit accidental changes but do not eliminate selection bias or detection error.

The final conclusion must distinguish supported automated indicators from validated cases, and reported uncertainty from an actual absence of evidence. The full supporting discussion is maintained in `THREATS_TO_VALIDITY.md` and the RDRs.

Conclusion

We investigated whether modified or reused agent skills contain security-sensitive operations absent from a related earlier observed artifact. The study combines content similarity, evidence for plausible earlier/later ordering, comparison of instructions and available supporting files, and eleven defined risk indicators.

Across three non-overlapping Sprint 3 trials, the automated analysis identified supported indicators in **63 of 300 sampled families** (**21.0%**). The most widely observed changes concerned local script execution and external code or dependency acquisition. These are useful review points because a change to a skill can change what an AI agent is instructed to run or obtain, even when the skill's name and general purpose remain the same.

The findings are **not** confirmed vulnerabilities or evidence of malicious intent. GitSkills does not establish verified copying histories, some evidence is incomplete, and independent manual validation of relationships and indicator judgments is still in progress. The sample results also cannot be generalized to every GitSkills family.

Our contribution is a reproducible way to identify and explain candidate changes while preserving uncertainty about their source and meaning. The final assessment of detection accuracy and individual cases will depend on the team's completed manual reviews.

References

Agent Skills. n.d. *Specification*. Accessed October 26, 2026. https://agentskills.io/specification.

Destefanis, Giuseppe, Daniel Graziotin, Matteo Vaccargiu, and Marco Ortu. 2026. *GitSkills: A Dataset of Agent Skills on GitHub*. Version 1.0.0. Zenodo. https://doi.org/10.5281/zenodo.21875637.

Huang, Jinjun, Jialun Cao, Junxiao Han, Zhonghao Jiang, Shing-Chi Cheung, and Zhongxin Liu. 2026. *How Agent Skills Evolve: A Large-Scale Empirical Study of Skills on GitHub*. https://arxiv.org/abs/2611.04456.

Liu, Yi, Zhihao Chen, Yanjun Zhang, et al. 2026. ""Do Not Mention This to the User": Detecting and Understanding Malicious Agent Skills in the Wild." *35th USENIX Security Symposium* (*USENIX Security 26*), 1727–46. https://www.usenix.org/conference/usenixsecurity26/presentation/liu-yi.

Liu, Yi, Weizhe Wang, Ruitao Feng, et al. 2026. *Agent Skills in the Wild: An Empirical Study of Security Vulnerabilities at Scale*. https://arxiv.org/abs/2601.10338.

Appendix A Dataset construction and preparation

Candidate selection

GitSkills is imported from the released SQLite database into a separate DuckDB database. The following query from `sql/create_analysis_tables.sql` materializes the comparison candidates without changing the source `artifacts` table.

```
CREATE TABLE artifact_groupings AS
WITH filtered AS (
    SELECT
        a.id AS artifact_id,
        a.repo_id,
        a.name,
        regexp_replace(
            lower(trim(a.description)),
            '\s+',
            ' ',
            'g'
        ) AS normalized_description
    FROM artifacts AS a
    WHERE a.frontmatter_valid = 1
      AND a.body_chars > 50
      AND a.content IS NOT NULL
      AND a.name IS NOT NULL
      AND a.description IS NOT NULL
),
eligible_names AS (
    SELECT
        name
    FROM filtered
    GROUP BY name
    HAVING COUNT(*) >= 2
),
groups AS (
    SELECT
        f.name,
        f.normalized_description,
        COUNT(*) AS artifact_group_size
    FROM filtered AS f
    JOIN eligible_names AS e ON e.name = f.name
    GROUP BY
        f.name,
        f.normalized_description
),
group_ids AS (
    SELECT
        row_number() OVER ()::BIGINT AS id,
        name,
        normalized_description,
        artifact_group_size
    FROM groups
)
SELECT
    g.id,
    f.artifact_id,
    f.repo_id,
    f.name,
    f.normalized_description,
    g.artifact_group_size,
    NULL::BIGINT AS sibling_file_count,
    NULL::VARCHAR AS sibling_content_sha
FROM filtered AS f
JOIN eligible_names AS e ON e.name = f.name
JOIN group_ids AS g ON g.name = f.name
 AND g.normalized_description =
  ↪ f.normalized_description;
```

Candidates require valid frontmatter, more than 50 body characters, available content/name/description, and at least two qualifying occurrences with the same exact name. These conditions select analyzable comparison candidates; they do not establish safety, reuse, or common ancestry. Short, malformed, metadata-poor, and singleton skills are excluded.

Descriptions are lowercased, trimmed, and whitespace-normalized for subgrouping. Family analysis compares across description subgroups within the same exact name, preserving candidates whose descriptions changed. The repeated-name requirement counts occurrences, including identical copies, rather than distinct content or repositories.

Preparation workflow

The activity diagram connects candidate preparation to the end-to-end cohort analysis. Preparation counts come from the October 15, 2026 local audit; the three Sprint 3 samples are drawn from a separately screened subset of this candidate pool.

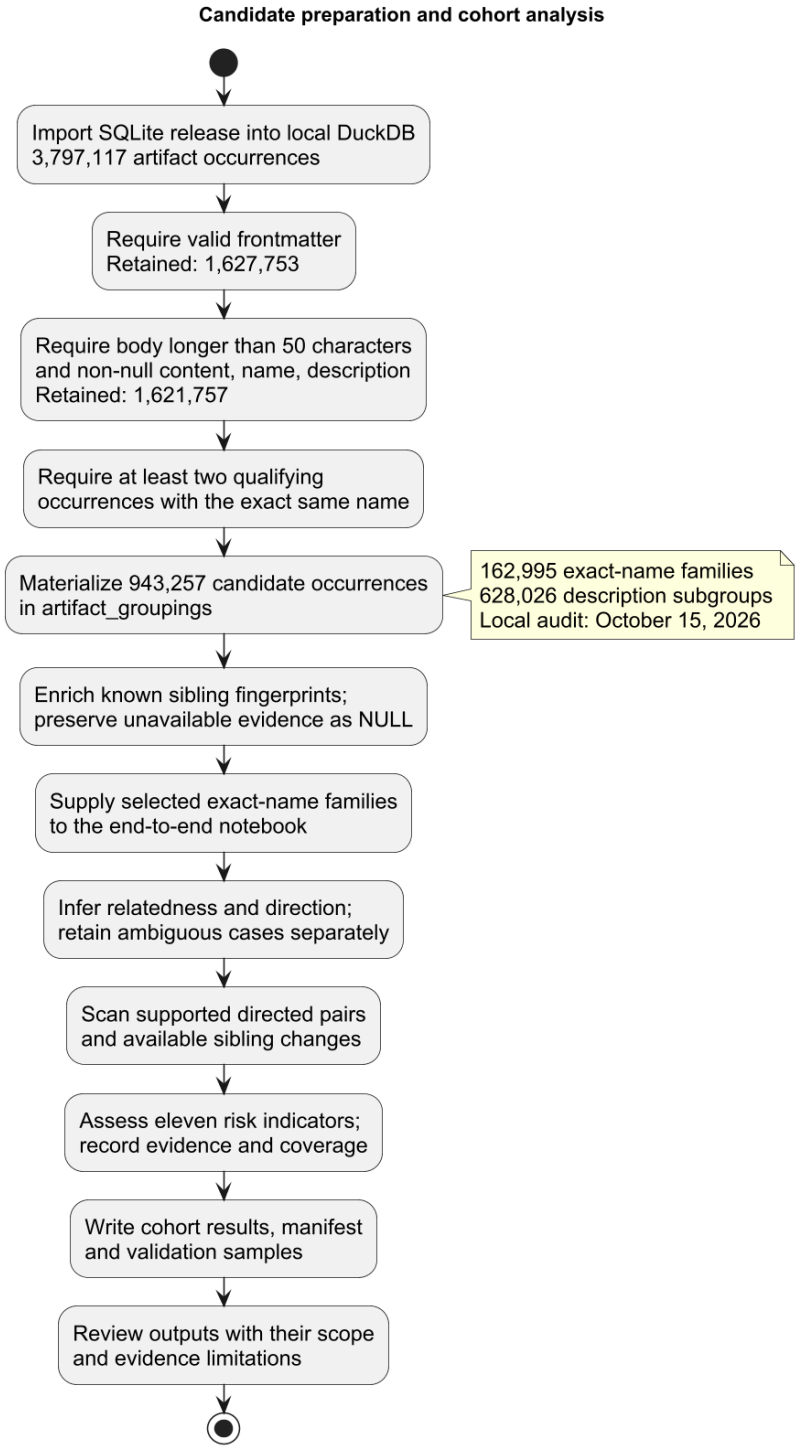


Figure 4: Candidate preparation and the end-to-end cohort analysis workflow.

Selection audit and identity

The audit retained 943,257 occurrences in 162,995 exact-name families. These are **not** the final family-selection pool or results denominator. The Sprint 3 family picker excluded names unsuitable for one-name-per-line selection, screened distinct main-file/sibling hash states, and checked for at least one usable primary directed comparison. Its sampling reports list 162,461 candidates at the preliminary selection stage; **not all of these candidates pass relationship validation**. The final analysis covers 300 randomly selected, validated families and 6,570 loaded artifact occurrences across three non-overlapping trials.

Artifact and description-group numbers are local to the prepared database. The query assigns group numbers without an explicit ordering, so cross-build reconciliation must use the dataset snapshot, repository/path identity, and content hashes. The sibling fields begin as null placeholders; subsequent enrichment distinguishes known empty bundles from unavailable or incomplete sibling evidence. Null must not be interpreted as an empty bundle.

Sprint 3 sample audit

The following counts come from the saved `sampling_report.json` files. A rejection means that the candidate failed the family picker's relationship validation, **not** that it contained no security-sensitive behavior. The picker replaced rejected candidates in the same stratum.

Each trial retains the same 20/55/20/4/1 size allocation described in Section 5. The sampling reports preserve the rejection reasons, source database identity, thresholds, excluded `families.txt` paths and checksums, and exact selected names. The 162,461 preliminary candidates are **not** an eligible-population denominator for estimating risk prevalence, because a validated primary directed relationship is required to enter the study.

Appendix B Pipeline implementation

Detection rule registry

The saved Sprint 3 manifests identify **15 active rules across six categories**. These rules recognize candidate evidence, while the eleven risk indicators (RIs) in Section 6 assess whether *changes* in that evidence satisfy specific criteria. A new rule match, increased rule count, or changed command text alone does not establish an RI.

The executable definitions are in `src/gitskills/rules/registry.py` and the category modules. In particular, RDR-012 distinguishes explicit package installation (`EXT-002`) from manifest-based dependency installation (`EXT-004`), while `EXT-003` detects staged downloaded-file execution. Overlapping rules can describe the same command, so individual matches are not independent events. Leading `SKILL.md` frontmatter is masked for behavioral analysis; sibling-file content is scanned separately when eligible and available. Dataset instructions are never executed.

End-to-end workflow and saved policy

For each selected family, the pipeline loads all qualifying artifacts, forms content variants and similarity clusters, evaluates the direction hierarchy, selects primary predecessors, and scans the directed pairs. It extracts evidence from both skill bodies and eligible added or changed sibling files, compares the base and derived evidence, applies RI-001 through RI-011, and records uncertainty and missing content. Cached family scans are reused only when their source, implementation, rule, and policy fingerprints are compatible.

Direction priority is declared source, direct chronology, equivalent-peer chronology, repository-creation boundary, then asymmetric containment. These are inference rules, not proofs of copying. Additional directed edges exist in the analysis method but **were not scanned for the three Sprint 3 result sets**. Ambiguous directions and equivalent peers remain available for relationship review without being treated as evidence that a behavior was introduced.

Main-file and sibling-change classification

Similarity equivalence does not require the same byte hash, so a `siblings-only` comparison can still contain textual differences in `SKILL.md`. Added or changed sibling **files** with available nonempty content are eligible for static scanning, regardless of extension. Unavailable content is reported as missing; paths, filenames, and directory changes alone are not treated as the behavior of an unknown script.

Relationship and scanning diagrams

The first diagram shows how the analyzer separates relatedness, direction, equivalent peers, and selected versus additional edges. It is a **conceptual diagram**; only selected primary edges enter the Sprint 3 security counts. The second diagram outlines the family-scanning component, including conditional sibling analysis. Risk-indicator assessment and cohort aggregation follow the underlying evidence comparison.

Cumulative selection stage	Retained artifacts	Excluded at this step
All source artifact occurrences	3,797,117	-
`frontmatter_valid = 1`	1,627,753	2,169,364
`body_chars > 50`	1,621,757	5,996
`content IS NOT NULL`	1,621,757	0
`name IS NOT NULL`	1,621,757	0
`description IS NOT NULL`	1,621,757	0
Exact name occurs in at least two qualifying rows	943,257	678,500

Table 8: Candidate-selection audit for the prepared GitSkills snapshot

Sample	Seed	Families selected	Candidates rejected during drawing	Previously selected families excluded
Trial 1	5801	100	239	0
Trial 2	5802	100	288	100
Trial 3	5803	100	247	200

Table 9: Sprint 3 sample seeds and family-selection audit

Category	Active rules	What they recognize	Main limitation
Command execution	`CMD-001`, `CMD-002`	Shell-command blocks and recognized command invocations	A code example is not proof of execution.
Network access	`NET-001`, `NET-002`	HTTP(S) URLs and recognized network clients	A documentation link alone is not a network operation.
File-system access	`FS-001`, `FS-002`, `FS-003`	File commands, writing APIs, and reading commands/APIs	Ordinary file use is not automatically sensitive or destructive.
Credentials	`CRED-001`, `CRED-002`	Secret-related references and recognized credential files	A name or placeholder does not prove access to a secret.
External code and dependency acquisition	`EXT-001`, `EXT-002`, `EXT-003`, `EXT-004`	Direct download-to-shell, explicit package installs, staged download-and-execute, and manifest-based dependency installs	A legitimate installation may match; multi-step behavior may be incomplete.
System modification	`SYS-001`, `SYS-002`	Privilege elevation and system-changing commands	Administrative instructions can be legitimate.

Table 10: Fifteen detection rules and their coverage limits

Setting	Sprint 3 value
Main study	Three non-overlapping, 100-family stratified samples
Family seeds	5801, 5802, 5803
Manual review seed	580
Edge selection	`primary` only
Shingle width	5 tokens
Minimum containment	0.80
Minimum Jaccard	0.30
Minimum shared shingles	5
Containment direction margin	0.10
Detection rules / risk indicators	15 / 11
Source code revision	`ae69732ed8988ad7066ff351848f7d2e8ad311a1`

Table 11: Pinned analysis configuration for Sprint 3

Change type	Meaning
`skill-only`	Main skill text differs; known sibling state is equivalent.
`siblings-only`	Main skill text is equivalent **under the analyzer's similarity representation**; known sibling state differs.
`skill+siblings`	Main skill text and known sibling content both differ.
`equivalent`	Both main-file and known sibling states are equivalent.
`unknown`	Missing or uncertain sibling state prevents reliable bundle classification.

Table 12: Bundle change classifications

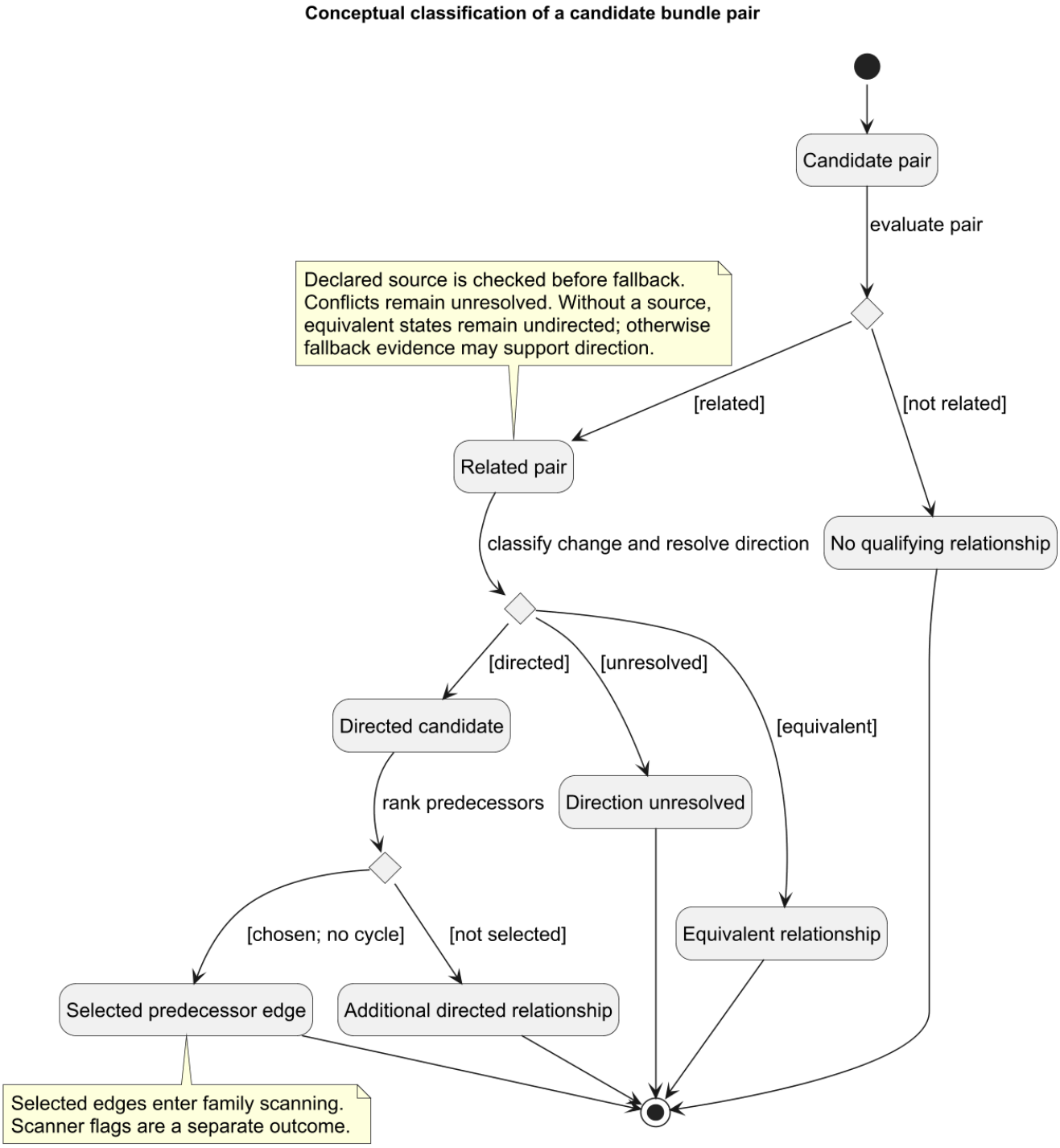


Figure 5: Conceptual UML state diagram for relationship classification. Scanner flags are separate from lineage outcomes.

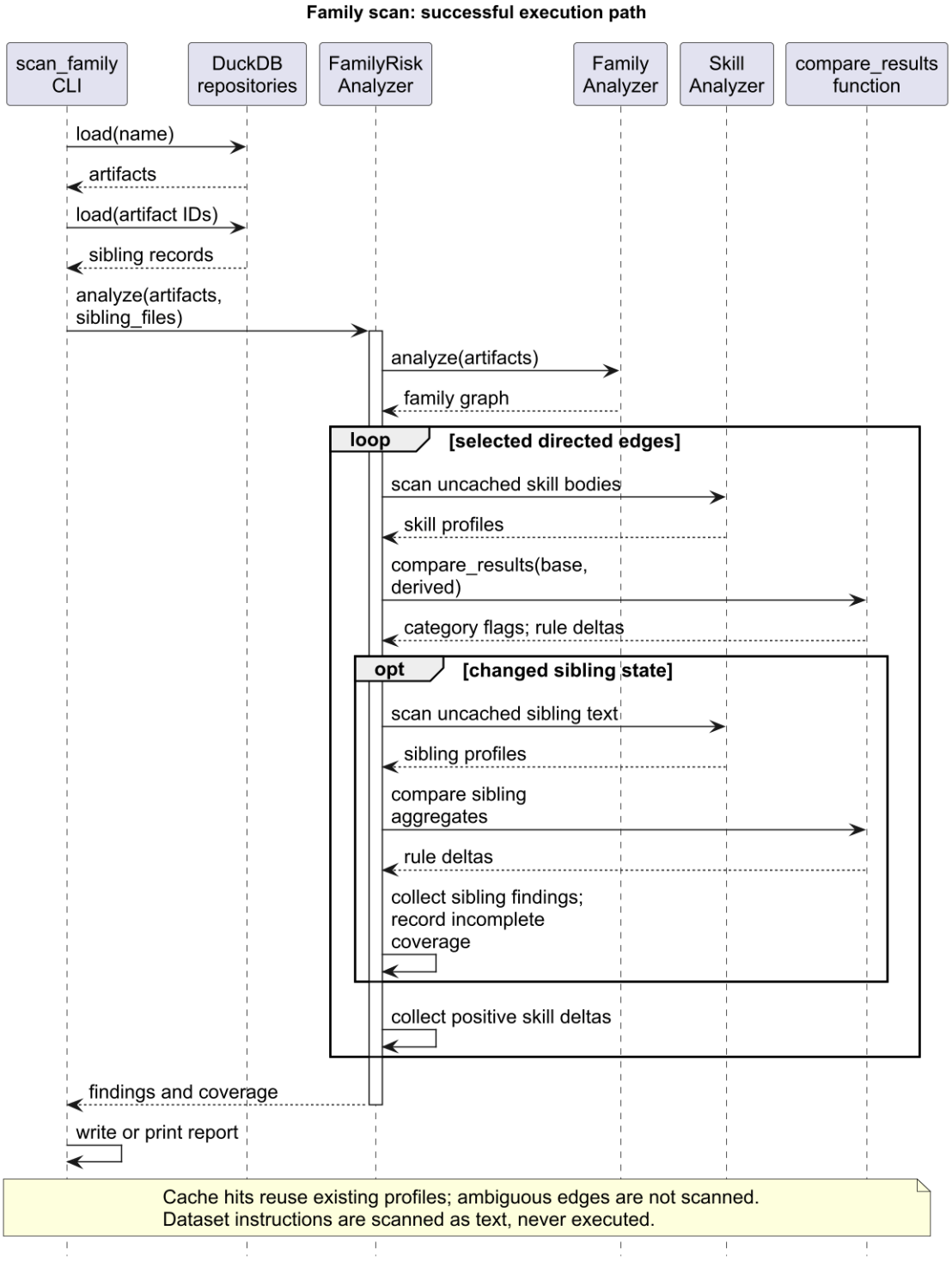


Figure 6: Family scanning and conditional sibling analysis; shown as a component-level summary rather than the complete research aggregation.

Appendix C Validation scope and status

Each of the **three Sprint 3 trials** generated **30 directed-comparison review cases** (five from each of six strata) and **15 relationship-review cases** (five from each of three strata), using review seed **580**. The total is **90 directed cases and 45 relationship cases**. These samples come from the saved results for each separate 100-family trial; they are **not** additional data collected for the final analysis.

Directed review covers supported indicators, uncertain indicators, flagged comparisons without supported indicators, coverage-limited comparisons, sibling changes, and unflagged directed comparisons. Relationship review includes equivalent, ambiguous, and no-related-edge cases. The two reviews serve different purposes: relationship review examines the similarity and ordering decisions, while directed review examines extraction, source differences, RI judgments, and coverage.

Reviewers distinguish rule matches, extracted evidence, supported versus uncertain indicator decisions, missing evidence, and evidence for direction. A reported RI is a **false positive** when manual inspection does not support it under the defined indicator policy. A **false negative** requires a manually supported *in-scope* indicator that the scanner did not report as supported. A reviewer should not count a benign change outside RI-001 through RI-011 as a missed indicator merely because it is security-related in another sense.

At the time of the saved runs, the annotation templates and selected cases existed, but **no independently adjudicated Sprint 3 validation metrics were included** in the summaries. Manual review is ongoing. We therefore do not report precision, recall, reviewer agreement, or confirmed positive/negative counts yet. When adjudicated files are available, the pipeline can recompute the validation portion of each saved summary without changing the original sample or scanner outputs.

The stratified review samples help characterize error types and coverage gaps, but their raw proportions should not be treated as population-wide accuracy estimates without a justified sampling calculation. Detailed review instructions are in `docs/instructions/manual_annotation_protocol.md`; the saved case lists and manifest checksums are linked in Appendix E.

Appendix D Additional Sprint 3 results

The tables below are calculated from the saved Sprint 3 `review/summary.json`, `review/families.csv`, `review/directed_pairs.csv`, and `review/direction_basis.csv` files. All indicator results are **automated** and pending independent human adjudication. Per-indicator counts overlap: one comparison may contain more than one supported indicator, and multiple comparisons may share artifacts.

Indicator frequency by trial

Each entry counts **primary directed comparisons** with the indicated supported RI. The final column counts **distinct families** across the three non-overlapping trials that have at least one such comparison.

Direction evidence and selected comparisons

The following categories are mutually exclusive per directed comparison. Counts of comparisons with RIs are *not* counts of verified copied-skill lineages.

Bundle change types

`unknown` describes an unavailable or insufficiently known sibling state, not a known unchanged bundle. Equivalent relationships are not counted as base-to-derived comparisons. A `siblings-only` classification uses main-skill equivalence under the similarity representation rather than requiring identical byte hashes; see Appendix B.

Family-size and cluster notes

Across 300 families, the analyzer produced **438 similarity clusters**. Families with multiple clusters were uncommon in the smaller strata (0 of 60 Low families; 10 of 165 Medium) and more common in the larger strata (19 of 60 Large; all 12 X-Large; all three Extreme). These clusters indicate relatedness under the project's token-shingle method, not verified independent origins.

For illustration, selected large families show different cluster structures:

The full per-family counts, coverage records, and selected comparisons remain in the three saved `review/` directories. They are not duplicated here because the detailed tables are best inspected directly as reproducible data files.

Appendix E Supporting repository material

The report uses the **three saved Sprint 3 trials**, not the earlier Sprint 2 exploratory cohort. The source code revision recorded in all three manifests is `ae69732ed8988ad7066ff351848f7d2e8ad311a1`; the manifests additionally retain the implementation fingerprint, active rules, dataset identity, and selected inputs. Results are not recomputed by rebuilding the report.

The saved research inputs and outputs are organized as follows (each trial has the same layout):

```
notebooks/end_to_end_pipeline/sprint3/
  samples1/families.txt
  samples1/sampling_report.json
  samples2/families.txt
  samples2/sampling_report.json
  samples3/families.txt
  samples3/sampling_report.json
  results1/
    sample_manifest.json
    execution_summary.json
    review/summary.json
    review/families.csv
    review/directed_pairs.csv
    review/indicator_frequency.csv
    review/direction_basis.csv
    manual-review/directed-pairs/
    manual-review/relationship-pairs/
  results2/                  # same structure
  results3/                  # same structure
```

Indicator	Trial 1	Trial 2	Trial 3	All comparisons	Distinct families
RI-001	0	0	3	3	3
RI-002	0	0	0	0	0
RI-003	8	3	41	52	17
RI-004	0	0	0	0	0
RI-005	25	15	115	155	26
RI-006	57	7	95	159	10
RI-007	3	0	1	4	2
RI-008	8	4	6	18	8
RI-009	0	0	0	0	0
RI-010	33	23	145	201	39
RI-011	0	0	1	1	1

Table 13: Risk-indicator frequency across the three trials

Direction basis	All comparisons	With supported RIs	Incomplete evidence coverage
Declared source	10	0	1
Direct chronology	399	55	42
Equivalent-peer chronology	355	43	8
Repository-creation boundary	827	111	86
Directional containment	566	131	74
Total	**2,157**	**340**	**211**

Table 14: Direction evidence and coverage for selected comparisons

Change type	Comparisons	With supported RIs
`skill-only`	805	33
`siblings-only`	190	41
`skill+siblings`	804	242
`unknown`	358	24
Total	**2,157**	**340**

Table 15: Primary-comparison counts by bundle change type

Family	Artifact occurrences	Similarity clusters
`implement`	856	35
`mcp-builder`	543	11
`documentation`	508	19
`cso`	150	2

Table 16: Examples of family size and similarity clusters

The `sampling_report.json` files record sampling seeds **5801–5803**, stratum allocations, exclusions, and candidate rejections. The `sample_manifest.json` files record the selected families, policies, scanner and code identities, review sample checksums, and a source fingerprint. Research summaries and CSV files contain the automated outcome counts used in Sections 5–7 and Appendix D; audit and execution files provide coverage and processing status. The separate annotation files are reviewer-owned and remain distinct from automated predictions.

Two cross-trial charts in Section 7 are generated from the saved `families.csv` and `directed_pairs.csv` files using `report/figures/generate_sprint3_figures.py`. The script does not alter trial results or select new families.

The three sample manifests currently mark their approval reference as **provisional** (`sample_status: provisional`, no approval reference). The final report should not imply formal approval has been recorded unless the manifest or repository decision history is updated. A tagged release and final manual-validation record should be linked when available.

Appendix F Use of AI in development and report preparation

Project Group 8 used AI assistance during tool and pipeline development and in preparing this report. Development assistance included discussing implementation approaches, drafting and revising code, troubleshooting errors, and supporting documentation and verification work. These activities contributed to the components integrated into the end-to-end pipeline.

For the report, AI assistance supported organizing material, drafting and revising prose, condensing earlier documenta-

Repository resource	Purpose
End-to-end notebook	Reproduces cohort processing and result tables.
Pipeline README	Configuration, outputs, cache, and validation instructions.
Dataset preparation	GitSkills acquisition and DuckDB preparation.
Candidate-selection SQL	Generates the comparison candidate pool (Appendix A).
Sibling fingerprint SQL	Represents known versus unknown bundled-file content.
Sampling decision RDR-014	Family eligibility and stratified sampling.
Edge-selection decision RDR-013	Rationale for primary-only comparisons and family-level reporting.
Risk indicator definitions RDR-010	Operational scope of RI-001 through RI-011.
Threat model	Trust boundary, detection scope, and security interpretation limits.
Annotation protocol	Independent review and adjudication guidance.
Report build guide	Source sections, figures, and local report rendering.

Table 17: Reproducibility and supporting repository resources

tion, and checking consistency between the narrative and saved outputs. It also supported formatting and rebuilding the report. The numerical findings come from the saved pipeline artifacts; AI-generated explanations are not experimental measurements.

Project Group 8 remains responsible for the implementation, interpretation, and final submitted text. Generated code and prose require checking against project requirements, source evidence, and reproducible outputs. AI assistance does not establish the correctness of inferred relationships or reported indicators, and it does not resolve the validation limitations stated in the main report.